WEEK 2 · LESSON 11 OF 24 · HANDOUT

Cost, Speed and Reliability

Study time: **about 45 minutes** (reading, video and activity)

▶ **Watch:** https://www.youtube.com/watch?v=jmwLzX_ltbQ — open the lesson online to watch inside the course.

🎓 **Go deeper:** PyData Virginia 2025: Mastering LLMs — from Prompt Engineering to Agentic AI — https://www.youtube.com/watch?v=jmwLzX_ltbQ

🎓 **Go deeper:** posit::conf 2023: A hacker's guide to open source LLMs (Jeremy Howard) — https://www.youtube.com/watch?v=sYliwvml9Es

Your goal in this lesson: Measure the speed and cost of LLM calls, cut them with caching and good model choice, and handle errors, rate limits and timeouts so your app keeps working.

Three things users notice

1. **Speed (latency)** — seconds until an answer.
2. **Cost** — your bill grows with tokens × calls.
3. **Reliability** — the network fails, providers are busy, limits are hit.

```
import importlib.util, os, subprocess, sys

if importlib.util.find_spec("anthropic") is None:          # install once per Colab session
    subprocess.run([sys.executable, "-m", "pip", "install", "-q", "anthropic"], check=True)

try:
    from google.colab import userdata                     # Colab: key saved in the Secrets
panel (key icon)
    os.environ["ANTHROPIC_API_KEY"] = userdata.get("ANTHROPIC_API_KEY")
except ImportError:
    pass                                                    # not in Colab: set
ANTHROPIC_API_KEY in your environment
except Exception:
    print("Add a secret called ANTHROPIC_API_KEY in Colab's Secrets panel, then run this cell
again.")

import anthropic

client = anthropic.Anthropic()        # reads ANTHROPIC_API_KEY from the environment
MODEL = "claude-haiku-5-5"            # fast and low-cost; check docs.claude.com for current
model names

def ask(prompt, system=None, max_tokens=500):
    extra = {"system": system} if system else {}
    response = client.messages.create(model=MODEL, max_tokens=max_tokens,
                                      messages=[{"role": "user", "content": prompt}], **extra)
    return response.content[0].text
```

Measure first

```
import time

def timed_ask(prompt, **kwargs):
    start = time.perf_counter()
    response = client.messages.create(model=MODEL, max_tokens=kwargs.get("max_tokens", 200),
                                      messages=[{"role": "user", "content": prompt}])
    seconds = time.perf_counter() - start
    return response, seconds

response, seconds = timed_ask("Name three crops grown in Rwanda.")
print(f"{seconds:.2f} s, {response.usage.input_tokens} tokens in,
{response.usage.output_tokens} tokens out")
```

Output tokens dominate latency: the model writes one token at a time. Asking for **shorter answers** ("in one sentence", a lower `max_tokens`) is the easiest speed-up.

Track spending across your app

```
class UsageTracker:
    def __init__(self, price_in=1.00, price_out=5.00):     # example US$ per million tokens
        self.calls = self.tokens_in = self.tokens_out = 0
        self.price_in, self.price_out = price_in, price_out

    def add(self, usage):
        self.calls += 1
        self.tokens_in += usage.input_tokens
        self.tokens_out += usage.output_tokens

    def cost(self):
        return self.tokens_in / 1e6 * self.price_in + self.tokens_out / 1e6 * self.price_out

tracker = UsageTracker()
tracker.add(response.usage)
print(f"{tracker.calls} call(s), estimated ${tracker.cost():.6f}")
```

Log usage per feature and per user; set budgets and alerts in the provider console.

Cache repeated questions

Many apps receive the same questions again and again ("What are your opening hours?"). Do not pay twice:

```
import hashlib

cache = {}

def cached_ask(prompt, system=None):
    key = hashlib.sha256(f"{MODEL}|{system}|{prompt}".encode()).hexdigest()
    if key in cache:
        return cache[key], True
    answer = ask(prompt, system=system)
    cache[key] = answer
    return answer, False

for _ in range(3):
    answer, from_cache = cached_ask("What documents do I need to open a mobile-money account?")
    print("from cache" if from_cache else "new API call")
```

Only cache answers that do not depend on the user or on changing data. In production, use Redis or a database table with an expiry time.

Prompt caching for long, repeated instructions

If every call starts with the same long text (a policy handbook, a product catalogue), providers can cache that prefix so repeated calls are cheaper and faster. With Claude you mark the reusable part:

```
POLICY = "Company policy for customer support. " * 50       # imagine a long handbook here
response = client.messages.create(
    model=MODEL, max_tokens=200,
    system=[{"type": "text", "text": POLICY, "cache_control": {"type": "ephemeral"}}],
    messages=[{"role": "user", "content": "Can I get a refund for a failed airtime
purchase?"}],
)
print(response.usage)
```

On later calls with the same prefix, `usage` reports cached tokens, which are billed at a reduced rate (see the provider's documentation for minimum lengths and prices).

Choose the right model size

Task	Usually enough
Classify, extract fields, short replies	Smallest, fastest model
Everyday chat and writing	Mid-size model
Hard reasoning, long documents, complex code	Largest model

Test the small model first; move up only if your test set shows it is needed. A common pattern: a small model handles most messages and **escalates** hard cases to a larger model or a person.

Handle errors

```
import random

def ask_with_retry(prompt, attempts=4):
    for attempt in range(attempts):
        try:
            return ask(prompt, max_tokens=200)
        except (anthropic.RateLimitError, anthropic.APIConnectionError,
anthropic.InternalServerError) as error:
            wait = (2 ** attempt) + random.random()          # 1, 2, 4, 8 s... plus jitter
            print(f"{type(error).__name__}: retrying in {wait:.1f} s")
            time.sleep(wait)
        except anthropic.BadRequestError as error:
            print("Bad request — fix the code, do not retry:", error)
            return None
    return None

print(ask_with_retry("Say hello in Kinyarwanda."))
```

- **Temporary errors** (rate limits, network, server busy): retry with **exponential backoff** — wait longer each time, plus a little randomness so many clients don't retry together.
- **Permanent errors** (bad request, invalid key): don't retry; fix the cause.

- The official SDK already retries some errors; you can tune it: `anthropic.Anthropic(max_retries=3, timeout=30.0)`.
- Always have a **fallback**: a polite message, a simpler answer, or a hand-off to a person.

Key terms

- **Latency** — time until the answer arrives.
- **Rate limit** — the maximum requests or tokens per minute your account may use.
- **Caching** — reusing earlier answers or prompt prefixes.
- **Exponential backoff** — retrying after waits that double each time.
- **Fallback** — what the app does when the model cannot answer.

Remember this: Measure tokens and time, cache what repeats, use the smallest model that passes your tests, and treat errors as normal events with retries and fallbacks.

Name: _____ Date: _____

Activity

Task 1

Time the same question with `max_tokens=50` and `max_tokens=500` and "answer in one sentence" vs "explain in detail".

Task 2

Add a `max_size` to the cache that removes the oldest entry when it is full.

Task 3

Simulate an error by setting `client = anthropic.Anthropic(api_key="wrong")`. Which exception do you get, and should it be retried?

Task 4

Estimate the monthly cost of a bot answering 2,000 messages a day at about 400 input and 150 output tokens each, with the example prices.

Self-check

Think about your answers here, then take the quiz online — it is marked instantly and explains every answer.

1. What usually makes up most of an LLM call's latency?
   - A. Reading the API key
   - B. The system prompt
   - C. DNS lookups
   - D. Generating output tokens one by one
2. When is it safe to cache an answer?
   - A. Only for errors
   - B. When it doesn't depend on the user or on changing data
   - C. Always
   - D. Never
3. What does prompt caching help with?
   - A. Repeating a long identical prefix such as a handbook more cheaply and quickly
   - B. Changing models
   - C. Storing API keys
   - D. Removing tokens from answers
4. Which error should NOT be retried?
   - A. Network connection error
   - B. Rate limit
   - C. Bad request (invalid input)
   - D. Server temporarily busy
5. What is exponential backoff?
   - A. Retrying immediately forever
   - B. Switching off the app
   - C. Waiting longer after each failed attempt
   - D. Lowering the price
6. How should you choose a model size?
   - A. Start small and move up only if your tests show it is needed
   - B. Randomly
   - C. Always the largest
   - D. Always the cheapest regardless of quality

© WAMUGII TECH SOLUTIONS · wamugii.kento.rw · Original course material — for enrolled students.